

Examinationsuppgifter C-programmering (FYD401)

Version 2026



GÖTEBORGS UNIVERSITET

Innehållsförteckning:

I. Introduktion	3
II. Översiktligt innehåll.....	3
III. Examination Följande gäller:	3
KAPITEL 1: ALLMÄN KUNSKAP OCH GRUNDER I C	5
KAPITEL 2: FILHANTERING.....	15
KAPITEL 3: LÄNKNING	18

I. Introduktion

Välkommen till FYD401 C-programmering. Kursen har som syfte att lära dig grunderna i programmeringsspråket C. Detta kompendium kompletterar föreläsningarna och innehåller examinationsuppgifterna – de ditt kursbetyg kommer baseras på (se examination nedan), samt lite praktiska tips för att underlätta för både dig och oss. Förutom uppgifterna här rekommenderar vi att du själv löser ett antal övningsuppgifter från kurslitteraturen eller annan källa för att komma igång på bästa sätt.

II. Översiktligt innehåll

C-programmering fokuserar på grunderna i C och programmering i allmänhet. Mer detaljerat innehåll fås via föreläsningarna. Labdelen av kursen, och därmed examinationen, är uppdelad i tre delar:

- 1) Allmän kunskap och grunder i C.
- 2) Filhantering.
- 3) Länkning.

Varje del motsvaras av ett kapitel.

Varje del innehåller ett antal examinationsuppgifter.

III. Examination

Följande gäller:

- Kursen examineras helt med examinationsuppgifterna (alltså *ingen* tentamen).
- All examination sker under schemalagd laborationstid (alltså *ej* via inlämning).
- Examination sker kontinuerligt, i form av redovisning av hela uppgifter eller delkapitel – vänta *ej* till sista labtillfället med alla uppgifter!
- Vid examinationen visar du programmets kod och funktion, och svarar på frågor från labbhandledaren om hur uppgiften löstes samt för en diskussion om ev. förbättringar.
- Alla uppgifter har ett poängvärde och du får dessa poäng efter godkänd redovisning.
- Du **måste ha minst en uppgift per delkapitel godkänd** för att få minst betyget Godkänd.

Ditt slutbetyg på kursen bestäms utifrån följande kriterier:

Betyget Godkänd: är normalbetyget. Programmen skall vara välskrivna och kommenterade och givetvis lösa uppgiften på ett sätt som utnyttjar C och utvecklingsmiljön på ett bra sätt. Dina program ska hantera fel på ett korrekt sätt; som exempel: om användaren ska mata in heltal ska programmet styra så att inte decimaltal går att mata in och/eller ska ett felmeddelande genereras. **I allmänhet ska indata inte vara hårdkodad som konstanter i programmen.**

- Du måste minst kommit upp i **30** redovisade poäng och du **måste ha minst en uppgift per delkapitel godkänd** för att få Godkänd (G).

Betyget Väl Godkänd: här krävs lite mer av dig utöver kraven för godkänd;

- att dina program uppvisar en allmänt god programmeringsteknisk kvalitet
- att du följer anmodad kodstandard
- att du kommenterar koden väl
- att lösningarna meningsfullt utnyttjar finesser i både C och utvecklingsmiljön
- att de färdiga programmen är användarvänliga.

Du måste minst kommit upp i **40** redovisade poäng och du **måste ha minst en uppgift per delkapitel godkänd** för att få Väl Godkänd (VG).

Examinationens presentation i utvecklingsmiljön:

För att det ska bli lättare att redovisa ska du göra följande:

- Skapa EN arbetsyta 'workspace' för varje delkapitel.
- Skapa ETT projekt, på den aktuella arbetsytan, för varje deluppgift.
- Döp både arbetsytor och projekt på ett bra sätt.
- Presentera svar till 'essäuppgifter' i form av textfiler.

Kapitel 1: Allmän kunskap och grunder i C

Mål: Du ska lära dig använda utvecklingsmiljön, förstå grundläggande programmeringsstruktur och kunna skriva enkla C-program som läser in data från tangentbordet, utför beräkningar och presenterar resultatet med enklare formatering.

1.1 Utvecklingsmiljö, förbehandling, ASCII och kommentarer

Du måste ha minst en uppgift godkänd i det här delkapitlet!

1.1.1 'Hello world' – givetvis! - 1p

a) Skriv ett eget program som skriver ut

Låt oss göra lite matte:

*5 * 10 = 50*

Programmet måste skriva å och ö i kommandotolkens fönster och siffran "50" får inte finnas i källkod.

b) Förklara översiktligt vad stegen som sker från källkod till körbart program innebär:

- Preprocessor ("förbehandlaren")
- Kompilator
- Länkare

1.1.2 ASCII - 1p

Förståelse ASCII i C:

- a) vad är det hexadecimala värdet för tecknet "\$"?
- b) vilka två heltal utgör gränserna för alla versaler i LATIN-1?
- c) vilken datatyp används i C för att representera ASCII-tecken?
- d) skriv ett eget program som skriver ut:

I C kan vi göra en massa saker med escape-sekvenser:

vi har tabb

eller vi kan aktivera ett larm

(programmet ska larma vid exekvering)

OBS: tomrummet mellan "har" och "tabb" är ett tabulatorstecken.

1.1.3 Kommentarsyntax - 1p

Vilka av följande rader programkod är rätt formaterade? Rätta till de felaktiga och förklara varför/hur de var fel.

- a) `/* printf("Skriv ut..."); */`
- b) `// printf("Skriv */ ut...");`
- c) `printf("// Skriv ut...");`
- d) `printf("//Skriv ut...");`
- e) `printf(/* Skriv ut... */);`

1.2 Variabler, inläsning och utmatning

Du måste ha minst en uppgift godkänd i det här delkapitlet!

1.2.1 Inmatning och utmatning av tal – 1p

Skriv ett program som läser in ett heltal, multiplicerar det med två och beräknar hur många siffror resultatet har. Om användaren matar in felaktig data (dvs, inte ett heltal) skall programmet skriva ut ett meddelande och avslutas. Tips: använd funktionen `scanf`

1.2.2 Formaterad utmatning av tal – 1p

Skriv ett program som läser in två heltal, låt oss kalla dem P och Q. Använd funktionen `printf` för att producera följande utskrifter från programmet:

- Talet P på hexadecimal form,
- Talet P/Q på flyttalsform med exakt 3 decimaler,
- Talet $P\%Q$ på heltalsform
- Talet $Q*P$ på tiopotensform ("scientific notation"),
- Talet Q med nollpadding till totalt 9 siffror (ex: 747 skrivs 000000747)

Tips: Angående heltalsdivision, vad är skillnaden på P/Q och $(\text{float})P/Q$?

1.3 Styrutryck, operatorer och satser

Du måste ha minst en uppgift godkänd i det här delkapitlet!

1.3.1 Bitar – 1p

Skriv ett program som skriver ut antalet bitar i en int på din dator.

1.3.2 Operatorer - 1p

Vad blir resultaten av följande operationer?

Lös detta moment **utan dator** och visa steg för steg för varje uttryck.

Antag att följande definitioner finns:

int a=1, b=2, c=3;

a) $a \& \sim b \& \sim c$

b) $a \& \sim b \& c$

c) $a \&\& \sim b \& c$

d) $a \wedge b \& c$

e) $a \& b \wedge \sim c$

f) $a | b \& c$

Heltalen a, b och c består ut av **8 bitar**.

Tips: vad är den binära representationen av talen 1, 2 och 3?

1.3.3 Fler operatorer - 1p

Beräkna a för hand utan dator och visa steg för steg. Ta reda på vilka operatorer som beräknas först!

```
int a = (! (1==0) + (0==1) * 1) + (1 || 1 && 0) + (! 1 + 1);
```

1.3.4 **while**- och **case**-satser - 2p

Skriv ett program som låter användaren manipulera ett heltal genom en meny. Talet är från början lika med 0, och användaren erbjuds följande val i en meny:

- 1) Addera 1
- 2) Multiplicera med 2
- 3) Subtrahera 3
- 4) Avsluta programmet

Vid val 1-3 utförs operationen, och användaren kommer tillbaka till menyn. Vid val 4 avslutas programmet. Om användaren matar in något annat så skall programmet hantera det på något lämpligt vis. Tips: använd en **case**-sats för användarens val i menyn, och en **while**-sats för att repetera programmet tills användaren väljer att avsluta. Detta är ett vanligt mönster som du kan återanvända i andra program!

Exempel på utskrift:

Talet är 0, välj en operation:

- 1) Addera 1
- 2) Multiplicera med 2
- 3) Subtrahera 3
- 4) Avsluta programmet

>3

Talet är -3, välj en operation:

- 1) Addera 1
- 2) Multiplicera med 2
- 3) Subtrahera 3

```

4) Avsluta programmet

>1
Talet är -2, välj en operation:
1) Addera 1
2) Multiplicera med 2
3) Subtrahera 3
4) Avsluta programmet

>4
Programmet avslutas.

```

1.3.5 Växelkassa – 2p

När du handlar i en affär och betalar kontant får du kanske växel tillbaka. I regel får affärsbiträdet hjälp av kassaapparaten med att beräkna summan man ska få tillbaka, men inte alltid vilka sedlar och mynt som ska lämnas tillbaka. Skriv ett program som beräknar den växel biträdet ska ge tillbaka i samband med ett köp. Programmet ska, förutom att presentera beloppet kunden får tillbaka avrundat till närmsta 50-öring, även bestämma vilka, och antalet, sedlar och mynt. Kunden ska få så få sedlar och mynt som möjligt tillbaka. Programmet ska kunna ge växel tillbaka med sedlar av valörerna 1000, 500, 100, 50 och 20 samt mynten 10, 5, 1 och 50-öring. Du kan anta att det alltid finns tillräckligt antal av de sedlar och mynt som krävs.

1.3.6 Formatering och gränser - 1p

Skriv ett program som beräknar och skriver ut (med lämplig formatering) resultat från funktionen:

$$f(x) = 4x^3 + 3\sin^2(x) - 5x - 7$$

Programmet ska beräkna funktionen med följande gränser:

- i) Samtliga heltal x $[-15, 15]$
- ii) Samtliga x $[-1, 1]$ med en upplösning i x på 0.1

1.3.7 Sten, sax, påse – 1p

Använder din kunskap om C programmering för att skriva ett sten, sax påse spel: använder mot dator.

1.4 Funktioner och funktionsprototyper

Du måste ha minst en uppgift godkänd i det här delkapitlet!

1.4.1 Funktioner - 2p

Du skall nu skriva ett program som sträcker sig över flera källkodsfiler. I de flesta projekt organiseras kod i separata funktioner, i olika filer, efter syfte. Den som sedan vill använda sig av funktionerna *inkluderar* koden med hjälp av preprocessordirektiv. Du kommer nu skapa ett enkelt sådant *bibliotek* för konvertering mellan enheter.

- Skapa filerna `convert.h`, `convert.c` samt `main.c`. I `convert.c` skapar du en funktion som tar emot avstånd mätt i meter, och returnerar avståndet i millimeter. Placera en funktionsprototyp i `convert.h`. Skriv sedan ett enkelt program i `main.c` som använder sig av funktionen ur `convert.h`.
- Förklara hur instruktioner till förbehandlaren kan förhindra att samma deklarerationsinformation inkluderas dubbelt under kompilering
- Förklara skillnaden mellan följande två preprocessordirektiv:

```
#include <Test.h>
#include "Test.h"
```

1.4.2 Funktionsargument – pass by value - 1p

Skriv ett program med en funktion `dubblera`, som dubblar ett tal:

```
void dubblera(int x)
{
    x = 2*x;
}
```

Gör en `main`-funktion som 1) initierar ett heltal till något värde, 2) skriver ut det, 3) använder funktionen `dubblera` på det och 4) skriver ut talet igen.

- Vad ger ditt program för svar, och varför?
- Ge ett förslag på hur man kan göra en `dubblera`-funktion istället.

1.4.3 Omvandling – 1p

Skriv ett program som tar två datum och tider som indata, givna som år, månader, dagar, timmar, minuter och sekunder. Programmet skall beräkna differensen mellan de båda tidsangivelserna i sekunder. Det skall också ge differensen i sekunder, minuter, timmar, dagar, månader och år. Du kan ignorera skottår (och skottsekunder) i denna uppgift.

1.4.4 bit operationer – 1p

Skriv ett program med en funktion som skifta (eller flytta) ett teckenlöst heltal (`unsigned int`) ett antal steg åt vänster. Båda ges som indata. Givet två tal, `x` och `y`, så skall bitarna i `x` flyttas `y` platser åt vänster, och bitarna som förloras till vänster hamna till höger. D.v.s., bitarna som hamnar utanför de mest signifikanta placeras i de minst signifikanta positionerna.

1.5 Pekare och adresser

Du måste ha minst en uppgift godkänd i det här delkapitlet!

Pekarvariabler är helt enkelt heltal som kan tolkas som en viss position i datorns minne: en adress. Alla variabler finns lagrade någonstans i minnet, och har en adress. En pekare används för att referera till olika ställen i minnet. Har man en pekare kan man referera till det den pekar till med `*`-operatoren. Vice versa kan man med en variabel få veta dess adress med `&`-operatoren. Följande uppgifter syftar till att praktisera dessa

1.5.1 Pekare, avreferering och adresser – 2p

- a) Om vi har deklARATIONERNA

```
int heltal = 10;
int *pekare_till_heltal = &heltal;
```

Vilka av följande uttryck resulterar i en utskrift av talet 11? (och varför? Testa!)

- 1) `printf("%i\n", pekare_till_heltal + 1);`
 - 2) `printf("%i\n", heltal+1);`
 - 3) `printf("%i\n", *(&heltal)+1);`
 - 4) `printf("%i\n", *(pekare_till_heltal + 1));`
 - 5) `printf("%i\n", *pekare_till_heltal + 1);`
 - 6) `printf("%i\n", &heltal + 1);`
- b) Du har många gånger använt funktionen `scanf("%d", &heltal);` Vad gör `&`-tecknet egentligen så att `scanf` fungerar som den gör?
- c) Gör ett program med funktionen `dubblera` som i uppgift 1.4.2, men använd en pekare för att få det att fungera. `dubblera` har nu alltså deklARATIONEN `void dubblera(int *x)`

1.5.2 Flyttande pekare – 1p

Gör ett program med en meny liknande uppgift 1.3.4. Ditt program skall skapa en sträng i minnet:

```
char text[] = "Gothenburg";
```

samt en pekare som pekar på början av strängen:

```
char *my_pointer = text;
```

Låt nu användaren manipulera pekaren med menyvalen, och skriv ut vad pekaren pekar på. Pekaren får inte peka utanför det allokerade minnet. Exempel på utskrift kan vara:

```
Strängen är: "Gothenburg".
Pekaren pekar på "G", välj en operation:
1) Plus 1
2) Minus 1
4) Avsluta programmet

> 1
Strängen är: "Gothenburg".
Pekaren pekar på "o", välj en operation:
1) Plus 1
2) Minus 1
4) Avsluta programmet

> 1
Strängen är: "Gothenburg".
Pekaren pekar på "t", välj en operation:
1) Plus 1
2) Minus 1
4) Avsluta programmet

>4
Programmet avslutas.
```

1.5.3 Minnesadresser - 1p

Pekarvariabler är helt enkelt heltal som kan tolkas som en viss position i datorns minne: en adress. Anledningen till att vi inte bara använder heltal är att pekare har en egen matematik.

Skriv ett program som har tre pekarfält: en `int*`, en `char*`, och en `double*`, alla med 4 element. Programmet ska skriva ut adressen till alla element med hjälp av en för-loop och pekararitmetik. Programmet ska även skriva ut skillnaden mellan adresserna i det tredje och det andra elementet av alla fält.

Vad får du för resultat? Varför är det på detta sätt?

1.5.4 Fält - 1p

Att deklarera ett fält av variabler (en *array*) är i princip ett sätt att allokeras en viss mängd minne och få en pekare till det första elementet. Förklara vad som händer i de två programsegmenten nedan och ange speciellt var det finns fel som går igenom en kompilering men ändå inte gör det man kan tänka sig var det förväntade.

- a) **double** dblField [100];
 ...
 dblField = dblField + 10;
- b) **double** dblField [100], *pDblField;
 ...
 pDblField = dblField + 10;

1.5.5. Sortering och listor – 1p

a) Skriv ett program med en funktion som kombinerar två listor (inte *länkade listor*, se 1.6.1) genom att växelvis ta element från indata-listorna. Till exempel: med tanke på de två listorna [a, b, c] och [1, 2, 3], bör funktion ge [a, 1, b, 2, c, 3]. Den kombinerade listan skall vara ett objekt i minnet. De är inte tillräckligt att skriva ut resultatet.

b) Om man antar att de två listorna är i ordning, se till att ditt program skapar en kombinerad ordnad lista. Till exempel: med tanke på de två listorna [1, 2, 6, 9] och [3, 4, 5, 7] ska bli [1, 2, 3, 4, 5, 6, 7, 9].

1.5.6 Dynamisk allokering av minne – textsträngar - 2p

Med fält (arrayer) måste du bestämma dig för en storlek när du skriver din källkod. Med funktionerna `malloc` och `realloc` kan man allokera en valfri mängd minne och få en pekare till det. Allt minne man allokera på detta sätt måste man sedan manuellt frigöra med funktionen `free`. Använd dessa verktyg för att skapa ett program som accepterar indata av godtycklig längd (fältet skall inte allokeras i förväg). Efter inmatning har skett ska programmet dela upp den inmatade informationen i två separata textsträngar:

```
char *str1, *str2;
```

`str1` ska innehålla alla tecken med jämnt index från ursprungstexten medan

`str2` ska innehålla alla tecken från ojämna positioner.

Skriv ut de två resulterande strängarna till kommandotolkens fönster.

Tips: `malloc` med släktingar arbetar alltid med mängden i *byte*. Använd specialfunktionen `sizeof` för att ta reda på hur många byte en viss datatyp är. Exempelvis `sizeof(double)`, eller `sizeof(char)`.

1.5.7 Mer dynamisk minnesallokering – 1p

Skriv ett program som läser in ett valfritt antal ord. Orden skall lagras i strängar som allokerats för att få plats med varje ord, och pekare till varje ord skall lagras i en allokerad lista av pekare. Den första indatan till programmet är antalet ord som följer. Exempel:

Hur många ord vill du mata in? 5

Ge 5 ord nu:

Idag är en bra dag.

Här är dina ord:

Idag

är

en

bra

dag.

Använd malloc() och antalet ord för att skapa ett dynamisk fält med pekare till char. Eftersom varje element i listan är en pekare, blir pekaren till listan en pekare till pekare till char. Vid inmatningen av varje ord kan det lagras i en temporär sträng av fix storlek. Sedan allokeras dynamiskt minne för ordet och det kopieras till det minnesutrymmet.

1.5.8 Sortering – 2p

Skriv ett program som med olika funktioner sorterar 10 inmatade textsträngar först med avseende på längd och därpå i bokstavsordning, t.ex.:

a

aa

ab

aab

...

Presentera resultatet med lämplig formatering. Berätta för användaren vilka begränsningar som finns i programmet.

1.5.9 Textsträngar forts - 2p

Skriv ett program som avgör om en sträng är ett palindrom eller inte. Programmet ska kunna ta ett enstaka ord som inmatning men även meningar med upp till tio ord.

När användaren trycker 'Enter' ska programmet avgöra om det matats in ett eller flera ord samt presentera resultatet på ett tilltalande vis (tex i en tabell med två kolumner, se exempel).

Programmet *ska* använda dynamisk minnesallokering.

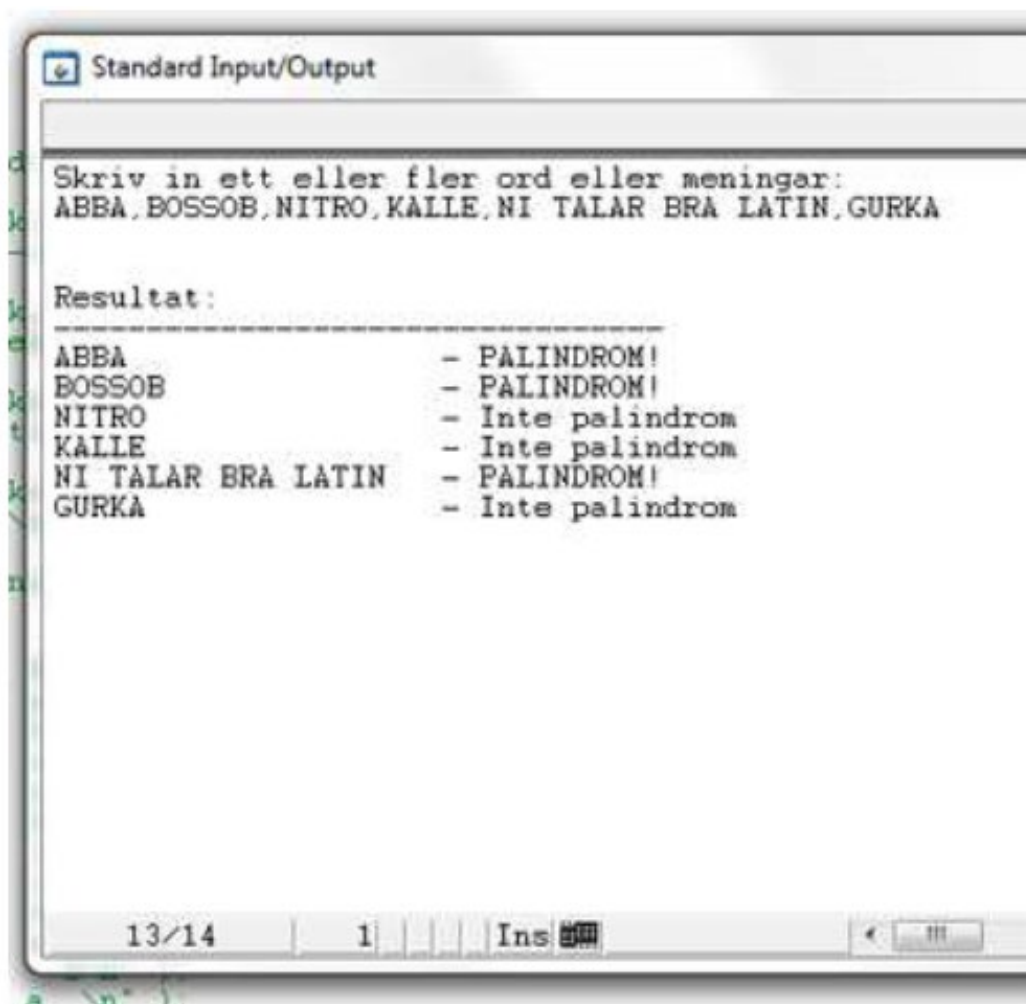
1.5.10 Fält av pekare - 1p

Skriv ett valfritt program som använder sig av ett fält av pekare till funktioner som alla returnerar ett 32-bitars heltal för att lösa en fritt vald matematisk uppgift.

Följande deklaration finns:

```
int (*ipFunks [3] ) (void);
```

Låt programmet skriva sina resultat till kommandotolkens fönster.



1.5.11 Säker kopiering – 2p

På nittioalet var en vanligt förekommande metod vid datorintrång att använda 'buffer overflow errors'. Det som utnyttjades var att om för mycket info ("för långa datapaket") skickades under vissa steg under t.ex. inloggning hamnade en del av paketet direkt i minnet och exekverades – intrång var därmed möjligt. Här ska information kopieras på ett säkert sätt som omöjliggör denna typ av attack.

Skriv ett program med en eller flera funktioner som kopierar information från en sträng till en annan. Funktionen ska dels acceptera godtyckliga teckensträngar av typen (**char ***) eller (**char []**) som är nollterminerade.

Då överlagrade funktioner inte existerar i C måste problemet med stränglängd lösas på annat sätt. Ni väljer fritt vilken metod som avgör huruvida den mottagna strängen är nollterminerad eller ej. All samlad funktionalitet som beskrivs i detta moment ska vara tillgängligt från ett och samma funktionsanrop. Er deklaration (och definition) ska se ut så här:

```
char *SafeCopyString ( char *dest, const char *origin, size_t count);
```

Funktionen ska returnera pekaren till den resulterande strängen och om det är nödvändigt ska den också signalera till användaren att ett fel inträffat under kopieringen. Tänk igenom vilka specialfall det finns. Det är *inte* tillåtet att använda standardfunktionen **strcpy ()** eller liknande funktioner. Testa programmet med alla olika aktuella parametrar.

1.6 Poster (structs)

Du måste ha minst en uppgift godkänd i det här delkapitlet!

En `struct` är ett sätt att gruppera flera variabler av olika typer till en ny, sammansatt, typ. Denna typ kan sedan användas som alla andra typer: man kan deklarerar variabler, man kan ha pekare till dem, man kan använda `sizeof` på dem, och så vidare.

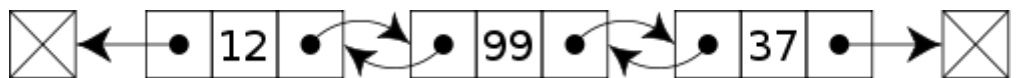
Många vanliga datastrukturer bygger på att man definierar en *nod* som innehåller dels data, dels pekare till andra likadana noder. På så vis kan man bygga upp en lista, eller ett träd.

Hantering av `struct` har lite speciell syntax (se kommentar nedan) så läs på om det och lös någon eller båda av följande uppgifter.

1.6.1 Länkad lista – 2p

Utgå från följande deklaration av en nod i en dubbelt länkad lista:

```
struct node {  
    double data;  
    struct node *next;  
    struct node *previous;  
};
```



Skriv ett program som har interaktiva funktioner för att

- Skapa en ny nod (dvs, kedjan med en ensam nod),
- lägga till en nod i slutet av kedjan av noder,
- ta bort en nod på en viss plats i kedjan av noder,
- gå igenom kedjan och skriva ut alla värden.

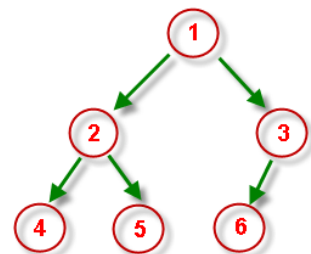
Och demonstrera denna funktionalitet. Ditt program måste allokera noderna dynamiskt, och frigöra minnet för dem efteråt.

1.6.2 Binärträd - 2p

Utgå från följande deklaration av en nod i ett binärträd:

```
struct node {  
    int data;  
    struct node *parent;  
    struct node *left;  
    struct node *right;  
};
```

Skriv ett program som skapar trädet i figuren till höger. Skapa en funktion som givet en rot-nod skriver ut trädet på något lämpligt sätt. Ditt program måste allokera noderna dynamiskt, och frigöra minnet för dem efteråt.



Kommentar om syntax: Vissa programmerare gillar att förenkla syntaxen för `struct` med hjälp av en `typedef`, såhär:

```
struct min_typ_t  
{  
    double data;  
    struct min_typ_t *next;  
};
```

```
typedef struct min_typ_t min_typ;
```

Efter det kan man använda "min_typ" utan att behöva skriva "struct" hela tiden. Vilket sätt ni väljer är en smaksak, det är ingen rätt eller fel i sammanhanget. Notera att i själva deklarationen av `struct` är inte `typedef` giltig ännu, så självreferenser måste alltid göras explicit.

Kapitel 2: Filhantering

Mål: Du ska förstå och kunna hantera filer; läsning, skrivning och kopiering. Du ska lära dig att utnyttja standardfunktioner och strömmar på ett bra sätt.

2.1 Standardfunktioner och filströmmar

Du måste ha minst en uppgift godkänd i det här delkapitlet!

2.1.1 Filer - 1p

Det kan ibland vara lite svårt att avgöra vad som finns i en fil och/eller i minnet. Här en övning som testar din förmåga att separera begreppen och att hålla tungan rätt i mun.

Konstruera ett program som först skapar tre matriser:

0.0 1.0	0.0 1.0	0.0 -1.0
-1.0 0.0	1.0 0.0	1.0 0.0

När matriserna är skapade ska programmet skriva ner var och en av dessa till en egen fil med namnen `matris1.bin`, `matris2.bin` och `matris3.bin`. Om filerna redan finns ska programmet skriva över dessa. Data skall skrivas till filerna i binärformat. Efter att samtliga matriser är sparade till fil ska programmet skriva ut matriserna som finns i minnet till **stdout** varpå de minnesresidenta matriserna skall nollställas. Programmet ska nu ånyo ladda in respektive matrisdata till rätt matris och sedan skriva ut den nyligen inlästa informationen till **stdout**.

2.1.2 Textanalys - 1p

Att söka i texter är ju ett mycket generellt programmeringsproblem.

Här ska du skapa ett program som kan avgöra om ett inmatat ord finns i en godtycklig fil. Om ordet hittas ska programmet meddela hur många bytes in i filen ordet hittades. Om ordet finns mer än en gång skall samtliga positioner presenteras. Utskrift ska ske till **stdout**.

2.1.3 Mönstermatchning - 1p

Att känna igen mönster i data är användbart för säkerhet, komprimering och kryptering.

Här ska ett program skrivas som kan känna igen mönster i 9 filer fördelade på 3 typer (och kataloger) med ändelserna `.1`, `.2` och `.3`. Filerna finns tillgängliga i Canvas (se mappen 'Exercises' under fliken 'Files').

Målet är att nya filer, som har samma mönster/format som de i de tre katalogerna ska kännas igen och identifieras som av formaten `.1` `.2` och `.3` via att öppna och analysera innehållet (ej ändelsen...). Vid examination är det bara handledare som kör ditt program. Du har inga möjligheter att ge ytterligare instruktioner.

2.1.4 Komprimering - 2p

Använd en godtycklig text på ca 2-3 sidor som du sparar som en fil och skriv sedan ett program som hittar de fem ord som förekommer mest frekvent. Programmet ska sedan ersätta dessa med varsin 'kod' som (helst) tar mindre utrymme i minnet. Skriv ut resultatet till en ny fil och meddela i kommandofönstret hur mycket plats som sparats. Skriv en funktion för att dekomprimera filen och kontrollera att originaltexten återfås.

2.1.5. Statistisk - 1p

Skapa ett program som öppna två filer med namn `2-1-5_I.txt` och `2-1-5_II.txt`. Filerna finns tillgängliga i Canvas (se mappen 'Exercises' under fliken 'Files'). Namnen av filer ska inte bli "hard-coded" i programmet. Programmet bör kunna analysera innehåll av filerna och räkna hur många gånger en viss bokstav eller en siffra förekommer i dem. Användaren ska kunna välja den bokstav eller siffra under programkörning. Dessutom bör program beräknas avståndet av förekomsten av

den bokstav eller siffra och analysera denna sträcka i termer av medelvärde och varians. Standardavvikelse definieras som:

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2}$$

Där N är antalet möjliga par, x_i är avståndet mellan ett par dvs. antal tecken emellan de två förekomsterna, och $\bar{x}$ är medelvärdet av avstånden för alla par.

Exempel: Om vi i sifferkombinationen "232642" letar efter tecknet '2', kommer vi ha tre par: 1:a och 3:e tecknet, 1:a och 6:e tecknet, 3:e och 6:e tecknet.

Om vi i detta exempel räknar avstånd i tecken får vi således avstånden: [2,5,3] och $\bar{x}=2.33$ med $\sigma=1.53$.

Programmet ska skapa två textfiler, som innehåller avstånden för de två indatafilerna i histogramformat.

2.1.6 ASCII konst – 2p

Skapa en textfil som består av 20 rader, var och en innehållande 30 heltal. Heltalen ska vara i intervallet 0-9 och separeras med mellanslag. Filen är en digital representation av en bild, där värdena 0 till 9 representerar ökande nivåer av mörker. Skriv ett program som läser innehållet i filen till ett 20x30-matrisfält av heltal. Omvandla den digitala representationen till en bild, där programmet använder värdena för att initiera ett 20x31-matrisfält av tecken, där ett 0-värde motsvaras av ett mellanslag, värdet 1 av en punkt, och så vidare, så att varje större tal representeras av ett tecken som har mer svärta. Du kan till exempel använda # för att representera 9. Sista tecknet (det 31:a) i varje rad bör vara ett *null*-tecken, vilket gör matrisen till ett fält med 20 strängar. Låt programmet visa den resulterande bilden (det vill säga skriv ut strängarna) samt lagra resultatet i en textfil.

Ett exempel på en indatafil kan se ut så här:

```
0000000000000000900000000000000000
0000000000000000900000000000000000
0000000000000000900000000000000000
0000000000000000900000000000000000
0000000000000000900000000000000000
0000000000000000900000000000000000
0000000000000000900000000000000000
0000000000000000900000000000000000
0000000000000000900000000000000000
0000000000000000900000000000000000
0000000000000000900000000000000000
9999999999999999999999999999999999
0000000000000000900000000000000000
0000000000000000900000000000000000
0000000000000000900000000000000000
0000000000000000900000000000000000
0000000000000000900000000000000000
0000000000000000900000000000000000
0000000000000000900000000000000000
0000000000000000900000000000000000
0000000000000000900000000000000000
0000000000000000900000000000000000
0000000000000000900000000000000000
0000000000000000900000000000000000
```

2.1.7 Sorting – 2p

Det är viktigt i C-programmering att förstå hur man ska skriva och använda generaliserade funktioner. Implementera ett alternativ till standardbiblioteks funktionen *qsort*, genom att använda insättningssorterings algoritmen. Se https://en.wikipedia.org/wiki/Insertion_sort.

Det är mycket vanligt att man måste handskas med arrayer som behöver vara sorterade. Däremot så är *qsort* en väldigt bra algoritm för att sortera listor som är helt oordnade, men mindre optimal för listor som nästan är ordnade. I detta fallet är insättningssortering mycket bättre.

Sorteringsfunktionen ska vara definierad på samma sätt som *qsort*, det vill säga

```
void isort(void *base, size_t nmemb, size_t size, int (*compare)(const void *, const void *));
```

Implementera funktionen i filen *isort.c*

Skriv också ett program som testar din algoritm för en array med 20 element.

Tips:

1. Du kommer att behöva använda pekar-aritmetik. Eftersom detta inte är tillåtet för void-pekare, konvertera datatypen (så kallad type cast) till char, vilket ju har storleken 1 byte.

2. För `double *p`, kommer elementet `p[n]` finnas på plats `((char *)p)+n*sizeof(double)`

3. Funktionen *memcpy()* är användbar vid implementeringen utav denna algoritmen.

4. *qsort*-funktionen:

```
void qsort(void *base, size_t nmemb, size_t size, int (*compar)(const void *, const void *));
```

base är pekaren till första elementet i den array som ska sorteras. *nmemb* är antalet element i arrayen. *size* är storleken på varje element och *compar* är en funktion som jämför två element.

compar funktionen deklareras:

```
int compar (const void* p1, const void* p2);
```

Och returnerar ett värde som är <0 om *p1* ska vara före *p2* i listan och ett värde >0 om *p2* ska vara före *p1*. Om deras värde är ekvivalent returneras 0.

Kapitel 3: Länkning

3.1 Statisk

Du måste ha minst en uppgift godkänd i det här delkapitlet!

3.1.1 Statisk länkning - teori - 1p

Förklara utförligt hur statisk länkning påverkar programmet m a p:

- Filstorlek
- Prestanda
- Kompatibilitet/Portabilitet

3.2 Dynamisk länkning

Du måste ha minst en uppgift godkänd i det här delkapitlet!

3.2.1 Dynamisk länkning i praktiken - 2p

Här ska du skriva ett program som på enklaste sätt visar på principen genom att två olika funktioners instruktionsförlopp endast ska vara tillgängliga genom två olika, dynamiskt länkade, bibliotek (som du själv byggt).

Programmet har följande specifikation:

- All utskrift sker till kommandotolkens fönster.
- Varje utskriftssteg följs av en radmatning och vagnretur.
- Vid programstart skrivs "Programstart"
- Programmet anropar sedan en funktion placerad i ett av de två dynamiskt länkade biblioteken. Denna funktion skriver ut "I .dll nummer ett."
- Programmet anropar sedan en funktion placerad i det andra dynamiskt länkade biblioteket. Denna funktion skriver ut "I .dll nummer två."
- Programmet avslutas.